Threadplane

Generative Interfaces

Let agents describe the interface. Keep your application in control.

Threadplane · Open source · Angular + React

Engineering guide · 2026 · Research reviewed November 7

threadplane.ai

Inside this guide

Practical boundaries, current APIs, and decisions to make before you ship.

01 Choose how much of the interface to generate

02 Design a catalog around user decisions

03 Match the rendering protocol to the host

04 Build an ingestion pipeline before rendering

05 Implement framework views with explicit profiles

06 Route actions through an accessible application contract

07 Test invalid surfaces and successful recovery

08 Dated capability and reference notes

For senior engineers and technical leaders

Framework-neutral architecture, with Angular and React examples. Published Angular adapters and native source-build bindings are distinct profiles; each example identifies its scope. Native packages remain private and are not npm installation instructions.

Research and repository API review: November 7, 2026. Model catalogs and upstream specifications are dated observations, not a promise of Threadplane support. Threadplane is free and open source; you own your backend, data and deployment.

Choose how much of the interface to generate

Generative interfaces are useful when the best presentation depends on information discovered during a task. They are less useful when they make a familiar action unpredictable. The engineering question is how much choice to give the model while preserving the application's interaction rules, evidence quality, and operational boundaries.

This paper follows a hypothetical invoice investigation. An assistant compares invoice INV-1042 with earlier records, explains discrepancies, and proposes an illustrative USD 480.00 credit. A generated evidence comparison may help the analyst understand the case. The final approval should retain a stable, recognizable contract, and a finance service must confirm the credit independently of what the generated surface displays.

Start with a fixed view when the job is known

A fixed result component is often the best first implementation. The application knows that an invoice search returns records, so it renders an authored table with predictable columns, sorting, and links. The model can decide which search to request without deciding the table's structure. This is already a useful agent-assisted experience.

Model-selected named views add a controlled degree of choice. The assistant might choose between a compact summary and a side-by-side comparison, with both implementations owned by the application. Bounded composition goes further: the model assembles permitted components and relationships. Each step adds expressive power and a larger validation and testing surface.

Approach	Appropriate use	Principal tradeoff
Fixed tool-result view	Stable operation with known output	Less adaptability, strong consistency
Model-selected named view	Several established presentations	Selection quality becomes an evaluation concern
Bounded component composition	Variable evidence and useful structural variation	More graph, state, action, and accessibility cases
Conventional authored workflow	Repeated, consequential decisions	Requires product design, minimizes generated ambiguity

A deterministic invoice-number lookup should remain a conventional lookup. Generating a fresh form for it on every request adds latency and uncertainty without improving the decision. The invoice comparison may justify more flexibility because the relevant differences vary from case to case.

Evaluate the interaction, not the spectacle

An attractive generated dashboard is not enough evidence to adopt the technique. Ask whether users find the important discrepancy faster, correctly distinguish evidence from inference, and reach the appropriate next action. Include the cases where generation fails or returns an unsupported structure. A fallback that lets the analyst continue matters as much as the successful layout.

Generation frameworks support different ways to produce structured surfaces, but those mechanisms do not decide product suitability. Establish the fixed-view baseline before testing greater freedom. Primary reference: json-render generation modes.

Design a catalog around user decisions

A component catalog is a vocabulary of permitted presentations. It should describe what the application can render and what each view means. A registry connects that vocabulary to actual Angular or React implementations. Keeping the two aligned is a maintenance responsibility, especially when saved specifications outlive a frontend release.

Prefer meaningful components over arbitrary building blocks

For the invoice workspace, an initial catalog might include EvidenceSummary, InvoiceComparison, MissingEvidence, and CreditProposal. These are conceptual application components, not claimed Threadplane exports. Their names describe jobs the user understands. A catalog containing only generic boxes, text, and buttons gives the model more ways to arrange content but fewer cues about correct interaction.

Specify the information required for each decision. An InvoiceComparison needs invoice identifiers, service periods, currencies, totals, and links to evidence. A CreditProposal needs the proposal revision, target invoice, amount, reason, and current decision state. Make missing information explicit. An empty amount should not render as zero, and an absent source should not become a reassuring citation icon.

Separate validation layers

A schema establishes structural expectations. Graph validation establishes that referenced elements exist and permitted relationships are respected. Business validation establishes that the referenced proposal and amounts correspond to authoritative records. Authorization establishes whether this user may see or act on them. Accessibility and usability require additional evaluation of the resulting interaction.

For example, a perfectly valid comparison can put the invoices in the wrong order and reverse the conclusion. A valid identifier can point to another tenant's record. A permitted button can carry a stale decision token. The catalog is an important constraint, but no single schema proves all the properties a product needs.

Treat descriptions supplied to the model as part of the interface contract. Explain when a component is appropriate and what it must not imply. The MissingEvidence view should communicate an unresolved question, not decorate an incomplete investigation as a finished result. Keep these descriptions concise enough that the model can use the catalog effectively.

Version meanings as well as names

Changing the meaning of a prop while retaining its name can silently corrupt old specifications. If a total previously represented a proposed credit and now represents an outstanding balance, visual similarity hides a substantive contract change. Version the catalog or provide an explicit migration.

Retain representative saved specifications in tests. Exercise them against both Angular and React implementations when both are supported, while allowing intentional differences in composition. The goal is consistent business meaning rather than identical markup. Catalog and schema documentation provide the structural foundation; the domain contract must be authored by the application team. Primary references: json-render catalogs and schemas.

Match the rendering protocol to the host

Several ecosystems use structured output to create interfaces, but their contracts are different. Choose the host and interaction model first, then select a compatible protocol and version. A payload described as JSON is not automatically acceptable to every renderer.

Distinguish native catalogs from embedded surfaces

json-render describes interfaces through specifications and application-defined components. In the local rendering examples, a specification names a root and element entries whose types resolve through a registry. A2UI uses its own messages, surfaces, catalog, and data model. They should not be treated as alternative names for the same wire object. Primary references: json-render specifications and A2UI specification.

MCP Apps associates interactive HTML resources with tools and renders them within a host-controlled environment. That can be appropriate when a tool provider supplies an embedded application. An application-owned Angular or React registry instead keeps component implementations in the product's own UI system. Neither approach removes the need for explicit action authorization. Primary reference: MCP Apps overview.

Decision	Native application catalog	Tool-associated embedded app
Who implements the visible components?	Product team and its design system	App resource author within host constraints
How does presentation evolve?	Catalog, renderer, and application releases	App resource and host compatibility
What requires explicit review?	Allowed types, props, actions, data bindings	Host permissions, resource behavior, tool interactions
What remains outside presentation?	Business authorization and execution receipts	Business authorization and execution receipts

Negotiate what the client actually supports

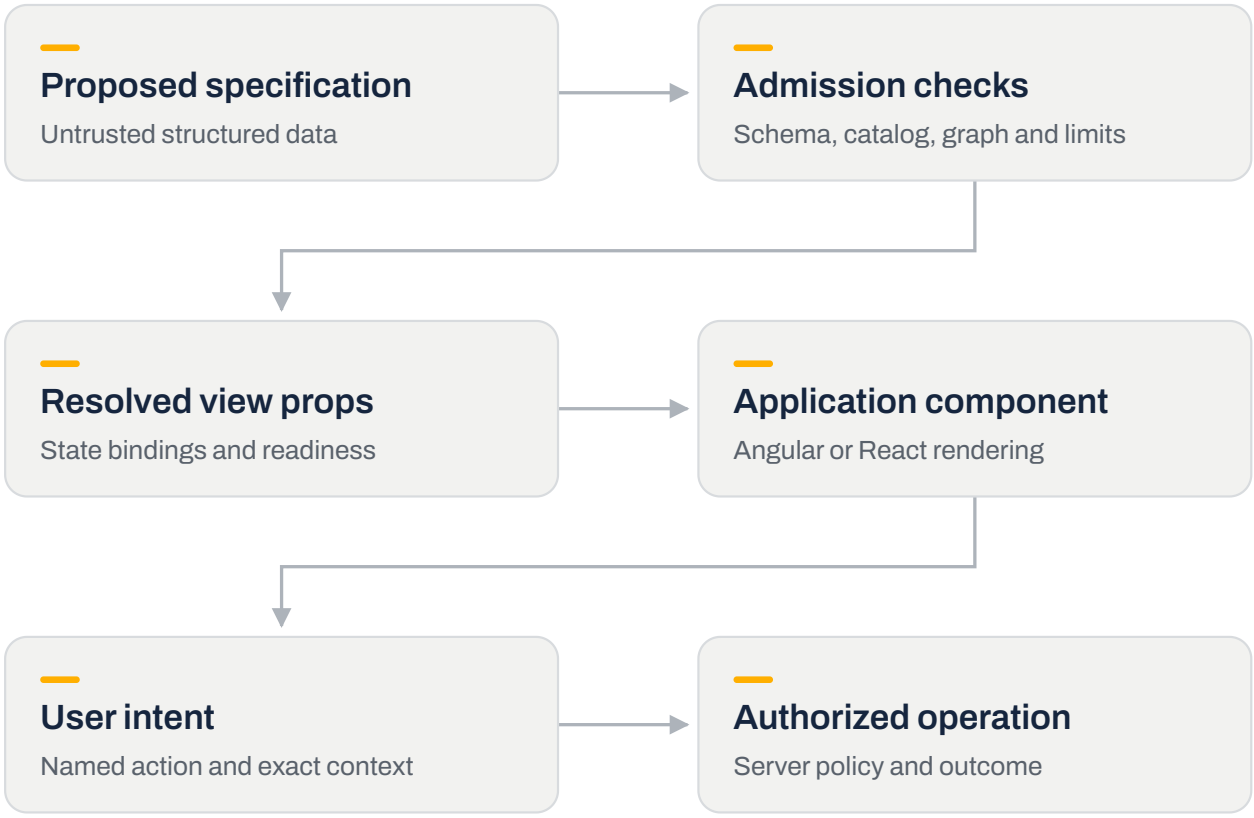
A backend should know which catalog and protocol version its client accepts. A mobile client with a smaller component set may need a simpler specification. A saved desktop surface may need migration before it can be shown on another client. Refusing an unsupported contract with a useful fallback is preferable to silently dropping the essential action or evidence.

The upstream version and the installed dependency are separate facts. The checked A2UI specification is v1.0, while Threadplane's local wire constant is v0.9. The checked json-render changelog reaches 0.22.0, while the local dependency is 0.16.0. Current upstream documentation may describe features outside the local integration. Verify support at the renderer and adapter boundary before promising them. Primary reference: json-render changelog.

Build an ingestion pipeline before rendering

The renderer should receive an accepted specification snapshot. Getting from network bytes to that snapshot is a separate pipeline with its own failure modes. Framing, parsing, ordering, validation, and data resolution do not become correct merely because a component can render a static example.

A proposal becomes an interface



A valid specification can describe a view, but cannot grant a permission. Incomplete or rejected proposals need an explicit fallback; accepted actions still cross the server authorization boundary.

In the invoice workspace, the producer proposes a comparison surface. The application validates its structure and catalog references, resolves permitted evidence data, and publishes a snapshot. Registered views display what is ready. User interaction becomes an application command, and the resulting authoritative state feeds a later observation. The diagram's boundary around execution matters: generating a button is not executing its action.

Accept complete snapshots before optimizing streams

Begin with a complete, validated specification and known state. This establishes whether the catalog and component contracts work. Then add streamed updates. Otherwise, a blank comparison might mean a parser problem, a missing element reference, an unresolved binding, or an unknown registry name, with no clear baseline for diagnosis.

For a patch-based producer, define the frame shape, sequence policy, and request association. Decide how duplicate frames are recognized and how gaps are handled. A patch targeting a missing element might require rejection or a fresh snapshot. Buffering it indefinitely can hide a broken stream and consume unbounded memory.

Do not assume that every stream consists of standard JSON Patch operations. The producer and consumer must agree on the specific protocol. Threadplane's rendering host receives the resulting specification; it is not a universal network client or incremental JSON parser. Upstream streaming utilities can inform a chosen pipeline without becoming an implicit feature of every host. Primary reference: json-render streaming.

Preserve the last accepted state

Maintain a distinction between a candidate update and the accepted surface. If an update introduces a missing root or an unsupported component, preserve the useful previous state and expose a bounded fallback or error. For a newly generated surface with no accepted state, present an explanation and a text or fixed-view alternative.

Associate updates with the current request and surface. If the analyst cancels one comparison and starts another, late frames from the first must not overwrite the second. The same rule applies to Angular signals and React state updates; the framework cannot infer which business request a frame belongs to.

Use stable element identities when the logical view persists. Replacing an entire surface for every new value can discard focus, expansion state, and user input. Conversely, keeping an identity while changing the element's meaning can preserve inappropriate local state. Define when an update is a revision of the same view and when it is a replacement.

Bound resource consumption

Set limits on element count, nesting, text size, update frequency, and unresolved references. A specification can be syntactically valid and still expensive to render. Limit diagnostic retention and ensure rejected content cannot trigger a feedback loop of repeated generation and mounting. These are application ingestion policies, not properties to assume from a valid schema.

Implement framework views with explicit profiles

Angular and React can share catalog concepts while using different component contracts. The smallest useful demonstration is a literal Text view. It isolates registry resolution from model output, transport, state binding, and action execution. Once it works, the application can introduce those concerns one at a time.

Published Angular registry

The following example uses the published Angular rendering profile. It registers an authored Text component and supplies a literal specification. It does not contact a model or demonstrate an authorized business action.

```
import { Component, input } from '@angular/core';
import type { Spec } from '@json-render/core';
import {
  defineAngularRegistry,
  RenderSpecComponent,
} from '@threadplane/render';

@Component({
  selector: 'app-label',
  standalone: true,
  template: `<p>{{ label() }}</p>`,
})
export class LabelComponent {
  readonly label = input('');
}

@Component({
  selector: 'app-generated-view',
  standalone: true,
  imports: [RenderSpecComponent],
  template: `<render-spec [spec]="spec" [registry]="registry" />`,
})
export class GeneratedViewComponent {
  readonly registry = defineAngularRegistry({ Text: LabelComponent });
  readonly spec: Spec = {
    root: 'greeting',
    elements: {
      greeting: { type: 'Text', props: { label: 'Hello, Angular.' } },
    },
  };
}
```

The Angular registry can associate a component with a fallback and a synchronous schema for readiness. Initial unresolved props can delay the real component. In the current implementation, readiness is a one-way gate for a mounted instance: later invalid or missing values do not automatically return it to the fallback. Components and ingestion validation must remain robust after admission.

An unknown component type has no registered implementation and does not produce a universal error card. The application should detect unsupported names before publishing a surface or provide a surrounding fallback. Containers must also render their child keys through the appropriate element renderer; listing children in a specification does not itself create their visual representation.

The Angular registry guide documents these local semantics. Keep shared rendering defaults in the supported application configuration scope, and choose an explicit store when the application needs to own data across surface updates.

Native React registry

The corresponding React example uses the private native source-build profile. It supplies a React registry and a literal specification to the native rendering surface. It demonstrates the same bounded choice of an authored view, without claiming that the package is publicly installable or that every Angular feature has an identical React counterpart.

```
import { RenderSpec, type ReactRenderRegistry,
  type RenderSpecData } from '@threadplane/react/render';

// Source-build profile; a literal, application-authored specification.
const registry: ReactRenderRegistry = {
  Text: ({ props }) => <p>{
    typeof props.label === 'string' ? props.label : 'Unavailable'
  }</p>,
};
const spec: RenderSpecData = {
  root: 'greeting',
  elements: {
    greeting: { type: 'Text', props: { label: 'Hello, React.' } },
  },
};

export function GeneratedView() {
  return <RenderSpec spec={spec} registry={registry} />;
}
```

The native React readiness entry uses an explicit synchronous `isReady` predicate. Its mounted admission is associated with the element and readiness contract; replacing that contract changes the lifecycle. This is a different API from Angular's synchronous schema configuration. Do not copy the Angular setup and assume only the template syntax changes.

The React registry guide is the appropriate implementation reference. Test readiness, missing types, data changes, and identity changes against the chosen profile. A shared product-level acceptance test can express the same expectation while its framework harness uses different APIs.

Keep state and structure separate

The comparison's element structure may remain stable while current invoice data changes. Local view state, such as expanded rows, should not require a model round trip. State expressions and computed functions, where supported, should be application-authored and bounded; they are not permission to execute arbitrary model-supplied code.

A render store is not durable business storage. Recreating a view may lose its local selections, and restoring a historical specification does not restore current authority. Bind consequential controls to current application state and make historical surfaces visibly historical. The same owner-versus-view distinction used for agent sessions applies here: observing a surface should not silently become ownership of an external operation.

Route actions through an accessible application contract

A user action can update local presentation, navigate to evidence, or request an external operation. These categories deserve different handling. Expanding a row can be immediate. Opening an invoice requires an authorized destination. Approving a credit must identify a current proposal and pass through the server's decision contract.

Give handlers narrow responsibilities

Register actions by purpose and validate their parameters. A handler for reviewing proposal P-7 should not accept an arbitrary URL or executable expression from the generated specification. The server should obtain the authoritative amount and account from the proposal record, not trust a client-supplied label.

In the worked example, the generated comparison may offer “Review proposed credit.” That opens an application-owned approval surface showing INV-1042, USD 480.00, the reason, and the relevant evidence. Approval then sends the decision token to the application boundary. The finance service's receipt establishes completion; a green success card generated before that receipt is merely an unsupported assertion.

If the proposal changes while the analyst is reviewing it, invalidate or refresh the decision. If the response is lost after approval, retain a checking state and reconcile the operation. A handler that returns successfully may only mean the request was accepted. Define what its result means before using it to drive success presentation.

Preserve keyboard and focus behavior

Generated surfaces must remain usable while they change. Prefer ordinary buttons, links, and form controls when they meet the need. Authored composite widgets need deliberate keyboard behavior; adding ARIA roles alone does not implement it. Keep focus visible and avoid moving it merely because another streamed element appeared. Primary reference: W3C ARIA keyboard-interface practices.

For the comparison, retain the analyst's position when a late evidence row arrives. When an approval panel opens through an explicit action, apply the focus behavior appropriate to that panel. When it closes, return the user to a sensible control. If an update removes the focused element, provide a predictable destination rather than leaving focus lost in the page.

Make progress legible without excessive announcements

A surface can announce a meaningful status change, such as evidence becoming ready, without reading every token or replacing focus. Ensure narrow layouts retain the invoice labels, amounts, and action controls. Dense comparison tables may need a deliberate responsive presentation so that evidence and its meaning remain associated.

These are design and verification responsibilities for both Angular and React components. A component's presence in a catalog does not certify accessibility. Primary references: W3C status messages and reflow.

Test invalid surfaces and successful recovery

Generative interfaces need tests for the distribution of outputs, not only a gallery of completed layouts. Begin with deterministic specifications that exercise each boundary. Then evaluate whether the producer selects and composes useful views for representative investigations.

Make failure cases reviewable

Injected condition	Expected application behavior
Unknown component name	Explain unsupported content and retain a useful alternative
Missing root or child reference	Reject the candidate without corrupting accepted state
Incomplete required props	Show the supported readiness fallback or bounded pending state
Invalid update after mount	Enforce ingestion policy and preserve a coherent interaction
Stale proposal token	Prevent the old surface from authorizing a new action
Late frame from another request	Ignore or reject it according to the stream contract
Excessive graph size	Stop at the configured resource boundary
Lost execution response	Reconcile operation status without claiming completion or replaying blindly

Exercise saved specifications from earlier catalog versions. Include a surface whose only unsupported element carries the essential evidence, not just a decorative label. Test a keyboard user who has focus inside a component while its data changes. Test a long supplier name, a missing currency, and a narrow screen with enlarged text.

Separate three kinds of success

Rendering success means the accepted specification produced the expected view. Interaction success means the user could understand and complete the intended task. Business success means the authoritative operation reached its defined outcome. A screenshot can support the first category, usability observation the second, and an integration test the third. None proves all three.

For the invoice scenario, acceptance should include correct evidence association, no actionable approval from incomplete data, preserved focus during updates, and a working fallback when generation fails. The action integration should reject stale decisions and reconcile OP-23 after a lost response. Model evaluation should examine whether the generated comparison helps distinguish duplicates from legitimate recurring charges.

Instrument rejection reasons and fallback use without logging unnecessary financial contents. A high rate of unknown components suggests catalog drift. Frequent missing evidence may indicate retrieval problems. Excessive regeneration may indicate that the product asks the model to solve layout choices better handled by a fixed view. Use these distinctions to improve the relevant boundary rather than enlarging the catalog reflexively.

Evaluation guidance can help structure representative tasks and grader calibration; the proposed acceptance cases here are application-specific. Primary reference: Anthropic, Demystifying evals for AI agents.

Dated capability and reference notes

Research checked on 7 November 2026. The chapter links are primary protocol and framework references or local implementation guides. The invoice workflow, catalog names, and acceptance policy are illustrative design proposals, not shipped business features or measured results.

Keep the compatibility record explicit

Surface	Baseline and limitation
Published Angular rendering	Version 0.3.7 profile; verify exported registry and host APIs
Native React rendering	Private 0.0.0 source-build profile; no public npm-install claim
Framework test baselines	Angular 21.1.6 and React 19.2.4 in the native repository profile
Current upstream frameworks	Angular 22 and React 19.3 in checked release sources
json-render	Upstream changelog 0.22.0; local dependency 0.16.0
A2UI	Upstream published v1.0; local wire constant v0.9
MCP Apps	A distinct embedded-app approach, not proof of native catalog support

Read the local Angular rendering introduction and React registry guide before adopting an example. Verify version-sensitive behavior against the installed package. Additional release references: Angular releases and React versions.